

Évaluation : Arbres binaires et arbres binaires de recherche

Question de cours

On note h la hauteur d'un arbre binaire et n sa taille.

En utilisant vos connaissances de cours, prouver que :

$$h \leq n \leq 2^h - 1$$

D'après exercice BAC

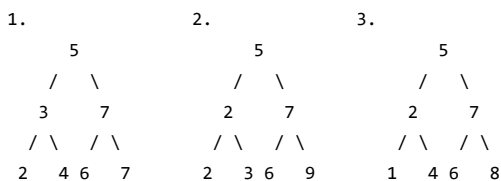
Rappel : Un arbre binaire de recherche (ABR) est un arbre binaire étiqueté avec des clés tel que :

- Les clés du sous-arbre gauche sont inférieures ou égales à celle de la racine;
- Les clés du sous-arbre droit sont strictement supérieures à celle de la racine;
- Les deux sous-arbres sont eux-mêmes des arbres binaires de recherche.

Partie A : Préambule

Question A.1

Recopier sur votre copie le ou les numéro(s) correspondant(s) aux arbres binaires de recherche parmi les arbres suivants :



Partie B : Analyse

On considère la structure de données abstraite ABR (Arbre Binaire de Recherche) que l'on munit des opérations suivantes :

Structures de données ABR

Opérations :

- `creer_arbre()` : renvoie un arbre vide
- `est_vide(a)` : renvoie `True` si l'arbre `a` est vide et `False` sinon
- `racine(a)` : renvoie la clé de la racine de l'arbre non vide `a`.
- `sous_arbre_gauche(a)` : renvoie le sous-arbre gauche de l'arbre non vide `a`.
- `sous_arbre_droit(a)` : renvoie le sous-arbre droit de l'arbre non vide `a`.
- `insérer(a, e)` insère la clé `e` dans l'arbre `a`

Question B.2.a

Dans un ABR, où se trouve le **plus grand** élément ? Justifier.

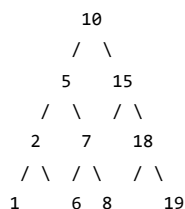
Question B.2.b

Pour rechercher une clé dans un ABR, il faut comparer la clé donnée avec la clé située à la racine. Si cette clé est la racine, La fonction renvoie vrai sinon il faut procéder récursivement sur les sous-arbres à gauche ou à droite.

En utilisant les fonctions ci-dessus, écrire une fonction récursive `rechercheValeur` prenant en arguments la clé recherchée et l'arbre ABR considéré. Cette fonction retourne un booléen (vrai ou faux) indiquant si la clé est présente dans l'arbre ou non.

Question B.3

On considère l'arbre ABR suivant :



- a. Donner le parcours infixe de cet arbre.
- b. Donner le parcours suffixe de cet arbre.
- c. Donner le parcours préfixe de cet arbre.
- d. Donner le parcours en largeur d'abord de cet arbre.

Club d'Informatique

Contexte

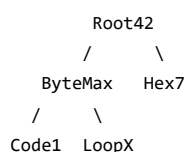
Un club de passionné·e·s d'informatique fonctionne de la façon suivante : pour être membre du club, à l'exception du fondateur ou de la fondatrice, il faut être parrainé·e. Chaque membre peut parrainer au maximum deux personnes.

On modélise ce fonctionnement à l'aide d'un arbre binaire dont les étiquettes sont les pseudonymes des membres du club.

Dans ce club, on distingue trois profils de membres :

- membre or : le membre a parrainé deux personnes ;
- membre argent : le membre a parrainé une seule personne ;
- membre bronze : le membre n'a parrainé personne.

Voici l'arbre P représentant les membres du club issus des parrainages de **Root42**, fondatrice du club. Par exemple, ByteMax a parrainé Code1 avant LoopX.



Question 1

- a) Indiquer la taille de l'arbre P .
- b) Recopier et compléter la définition de la fonction récursive `taille` qui prend un `arbre` binaire en paramètre et renvoie la taille de cet `arbre`.

```

def taille(arbre):
    if ... :
        return 0
    else:
        taille_gauche = taille(gauche(arbre))
        taille_droite = ...
        return 1 + ...

```

- c) Indiquer le type de la valeur renvoyée par la fonction `taille`.

Question 2

La fonction `membres` ci-dessous prend un arbre binaire `arbre` et une liste `liste_membres` en paramètres et ajoute, dans un certain ordre, les étiquettes de l'arbre à la liste.

```

def membres(arbre, liste_membres):
    if not est_vide(arbre):
        membres(gauche(arbre), liste_membres)
        membres(droite(arbre), liste_membres)
        liste_membres.append(racine(arbre))

```

- a) En supposant la liste `membres_p` initialement vide, écrire la valeur de cette liste après l'appel `membres(arbre_p, membres_p)` où `arbre_p` référence l'arbre P .

- b) Indiquer le nom du type de parcours d'arbre binaire réalisé par la fonction `membres`.

Question 3

Dans cette question, on s'intéresse aux profils des membres (or, argent ou bronze)

- a) Indiquer les étiquettes des feuilles de l'arbre P .
- b) Indiquer le profil des membres dont les pseudonymes sont les étiquettes des feuilles.
- c) Écrire la fonction `profil` qui prend un arbre binaire non vide en paramètre et renvoie le profil du membre dont le pseudonyme est l'étiquette de la racine de l'arbre. Par exemple, l'appel `profil(arbre_p)` doit renvoyer `'or'` qui correspond au profil du membre Root42, racine de P .

Question 4

Afin d'obtenir un tableau dont chaque élément est un tuple contenant le pseudonyme d'un membre et son profil, on propose la fonction `membres_profils` définie ci-dessous :

```
def membres_profils(arbre, liste_membres_profils):
    if not est_vide(arbre):
        membres_profils(gauche(arbre), liste_membres_profils)
        membres_profils(droite(arbre), liste_membres_profils)
        liste_membres_profils.append((racine(arbre), profil(arbre)))
```

On appelle cette fonction sur un arbre `arbre_2` et on obtient :

```
>>> liste_2 = []
>>> membres_profils(arbre_2, liste_2)
>>> liste_2
[('CodeX', 'bronze'), ('Bit0', 'argent'), ('Hex16', 'bronze'), ('ForMe', 'or')]
```

Dessiner l'arbre `arbre_2`.

Question 5

Chaque année, les membres versent une cotisation en fonction de leur profil :

- membre or : 30 €
- membre argent : 40 €
- membre bronze : 50 €

Écrire une fonction `cotisation` qui prend un arbre binaire et renvoie le total des cotisations reçues par le club. On pourra utiliser la fonction `membres_profils` de la question précédente.

Insertion et valeurs supérieures dans un ABR

Contexte

Dans cet exercice, la taille d'un arbre est le nombre de nœuds qu'il contient. Sa hauteur est le nombre de nœuds du plus long chemin qui joint le nœud racine à l'une des feuilles. On convient que la hauteur d'un arbre ne contenant qu'un nœud vaut 1 et la hauteur de l'arbre vide vaut 0.

On considère l'arbre binaire représenté ci-dessous :

```

      12
     /  \
    8    16
   / \  / \
  5 10 14
```

Question 1

- a) Donner la taille de cet arbre.
- b) Donner la hauteur de cet arbre.
- c) Représenter sur la copie le sous-arbre à droite du nœud de valeur **12**.
- d) Justifier que l'arbre est un arbre binaire de recherche.
- e) On insère la valeur **13** dans l'arbre de telle sorte que 13 soit une nouvelle feuille de l'arbre et que le nouvel arbre obtenu soit encore un arbre binaire de recherche. Représenter sur la copie ce nouvel arbre.

Question 2

On considère la classe `Noeud` définie de la façon suivante en Python :

```
class Noeud:
    def __init__(self, gauche, valeur, droite):
        self.gauche = gauche
        self.valeur = valeur
        self.droite = droite
```

- a) Parmi les trois instructions (A), (B) et (C) suivantes, écrire sur la copie la lettre correspondant à celle qui construit et stocke dans la variable `abr` l'arbre représenté ci-dessous.

```
    12
   /  \
  8    16
```

- (A) `abr = Noeud(None, 8, Noeud(Noeud(None, 12, None), 16, None))`
- (B) `abr = Noeud(Noeud(None, 8, None), 12, Noeud(None, 16, None))`
- (C) `abr = Noeud(Noeud(Noeud(None, 8, None), 12, None) 16, None)`

- b) Recopier et compléter la ligne 7 du code de la fonction `insérer` ci-dessous :

```
def insérer(v, abr):
    if abr is None:
        return Noeud(None, v, None)
    if v > abr.valeur:
        return Noeud(abr.gauche, abr.valeur, insérer(v, abr.droite))
    elif v < abr.valeur:
        return ...
    else:
        return abr
```

Question 3

La fonction `nb_sup` prend en paramètres une valeur `v` et un arbre binaire de recherche `abr` et renvoie le nombre de valeurs supérieures ou égales à la valeur `v` dans l'arbre `abr`.

Le code de cette fonction `nb_sup` est donné ci-dessous :

```
def nb_sup(v, abr):
    if abr is None:
        return 0
    elif abr.valeur >= v:
        return 1 + nb_sup(v, abr.gauche) + nb_sup(v, abr.droite)
    else:
        return nb_sup(v, abr.gauche) + nb_sup(v, abr.droite)
```

- a) On exécute l'instruction `nb_sup(13, abr)` dans laquelle `abr` est l'arbre initial de la question 1. Déterminer le nombre d'appels à la fonction `nb_sup`.
- b) L'arbre passé en paramètre étant un arbre binaire de recherche, on peut améliorer la fonction `nb_sup` précédente afin de réduire ce nombre d'appels. Écrire sur la copie le code modifié de cette fonction.